COMP 581: Introduction to Robotics

Fall 2026
Instructor: Prof. Ron Alterovitz <ron@cs.unc.edu>
Learning Assistant: Kamal Deep Vasireddy <kamalv@unc.edu>

Lab 3: Moving Around an Obstacle

Deadline

Tuesday, November 10, 2026, at the beginning of class. For the late policy, see the course syllabus. **Before the due time, each team should submit their team's code in Gradescope**; see specific instructions in the Code section below**.**

Submission Deadlines:

Optional Pre-Lab Reflection:	11/7 at 11:59 pm
Submit Code:	11/10 at 9:30 am
Optional Post-Lab Reflection:	11/13 at 11:59 pm

For all submissions, include the **names** and **PIDs** for each team member and include the **team number**. Add all team members to the submission using the **View or edit group** option in Gradescope.

The Objective

You will create a robot that determines the location of an unknown obstacle, traces the boundary of the obstacle, and returns to where it started.

Robot Design Requirements

Your robot must be built only using pieces from your LEGO Mindstorms kit. You should design your robot such that the dark gray center button may be pressed without moving the robot and such that pushing the button does not disrupt any of the sensors. Your robot should remain in one piece at all times, its diameter with respect to the ground should not exceed 40 cm, and its diameter shall not change substantially during the task. All measurements will be made with respect to the ***measuring point*****:** a distinct point of your choosing that is on the **frontmost surface** of your robot. More specifically, you will place your robot on the ground behind a starting line drawn on the ground, such that your chosen measuring point is **directly above a point on the starting line marked by a star**, and no part of your robot is permitted to be in front of the starting line. Your robot should be designed to accomplish the task autonomously without any human intervention (except for pushing the dark gray center button as required).

The Task

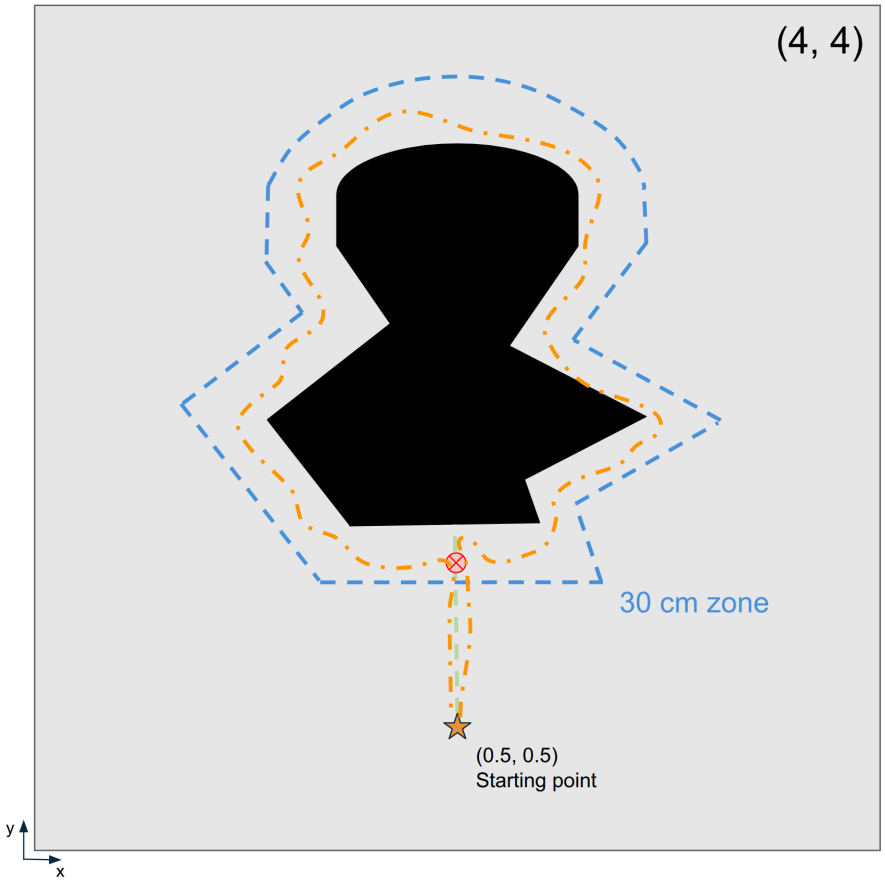
You will place your robot behind a starting line such that its measuring point will be on a specific *starting point* on the starting line. Directly in front of your robot will be an obstacle. The front wall of the obstacle (i.e., the wall of the obstacle facing your robot when it is at the starting point) will be located between 40 cm and 1.2 m away from the starting point. The point directly in front of your robot and 20 cm away from the obstacle's front wall will be known as the *hit point*.

To begin, you will push the dark gray center button on your robot. Your robot should then move straight forward until it locates the obstacle. To indicate that the robot has found the obstacle, it should beep when it is less than 30 cm from (or in contact with) the obstacle. The robot is now near the hit point. There is no need for your robot to stop, although it may. Your robot should then turn right and travel completely around the obstacle with its measuring point staying within 30 cm of the obstacle at all times. After the robot returns to the neighborhood of the hit point, the robot should turn away from the obstacle and return to the start point.

In this lab, you can assume that the obstacle is a closed shape consisting of straight and curved segments. It is not necessarily a convex shape. It will be made of bumpable, ultrasonic reflective walls that are at least 17 cm tall. The perimeter of the obstacle will be between 1.5 m and 7 m.

This lab is to be completed in a single run, without pressing any buttons after it has begun or picking up your robot (except as defined in the Points section below). All of the robot motion must be completed within a 4 meter square workspace without leaving the workspace. The starting point is at coordinate (2.0 m, 0.5m). The task must be completed in under 2 minutes.

In the figure below, we illustrate an example of a possible environment that complies with the rules. We note that the environment in your actual run may be different. The robot starts at the starting point (orange). The obstacle is shown in black. The wall shapes and the angles of any bend are unknown in advance. The red hit point marks the spot at which the robot should reach the obstacle and, after surrounding the obstacle, from which it should return to the start point. The dotted blue line represents a 30 cm distance to the obstacle. The yellow dashed line shows a possible robot trajectory that would receive full credit if completed in the allotted time, and assuming the robot stops at the start point.



Points

Logistics (10 points):
Turn in code on time: 10 points.
Performance (90 points):

- **Attempt**: 30 points. You will receive these points simply by having EV3 MicroPython successfully installed, showing up to the room on the lab due time, submitting your code on Sakai on time, having a robot that conforms to the design requirements, and having your robot attempt the course.
- **Find the hit point**: 15 points. Points will be awarded for finding the obstacle and beginning to follow its wall. You will receive these points if your robot beeps when it is within 30 cm of (or in contact with) the obstacle.
- **Percentage traced**: 30 points. Points will be awarded based on the percentage of the obstacle's perimeter that the robot successfully traced while staying inside the required distance from the obstacle wall. The percentage will be computed approximately as (the amount of obstacle boundary traced until the robot either violates the distance constraint to the boundary or returns to the hit point)/(the total perimeter of the obstacle).
- **Return to the start point**: 15 points. Points will be awarded for returning to and stopping at the start point. Points will be awarded based on accuracy (i.e., how close the robot's measuring point is to the start point when the robot stops).

Bonus:

- **New:** Bonus: Up to 3 bonus points can be earned for creativity. To earn these points, state your creative contribution at the beginning of your run. These creativity points will be added to your lab points up to a total of 100 points.

Penalties:

- There will be a 10-point penalty if, after crossing the start line, your robot exits the feasible workspace.
- **New: You may, at your discretion and only once, pick up your robot, press a single button, and put it down either at the start or where the robot was last in the 30cm zone. If you choose to do this, you will receive a 10-point penalty. Note that the 2-minute timer continues unchanged.**
- If your robot does not stop after 2 minutes, then your run will be terminated and no further points after 2 minutes will be earned.

Code

To complete this lab, you will use a subset of the EV3 MicroPython API:
https://pybricks.com/ev3-micropython/index.html
Important: You may only use the following modules from EV3 MicroPython API (covered in chapters 3-6 of the API document above):
ev3brick
ev3devices
parameters
tools
You may ***not*** import, use, or copy the "**robotics**" module, or any of its included classes or functions, in your program (covered in chapter 7 of the API document above).

A robot relying on a module not allowed as stated above will be disqualified. Also, any robot using Wi-Fi or Bluetooth communications in any way will be disqualified.

You must submit on **Gradescope** a zip file of your Python source code files. The zip file should include all the Python source code files you wrote (which should have a .py file extension). Please include the **names** and **PIDs** of all team members, along with the **team number** in the comments at the top of all included Python source files.

Optional: Lab Reflections

Optional reflections will award points towards the performance portion of the lab and are designed to prompt students to think systematically and critically about the tasks. You must have attempted a lab in order to receive credit for a post-lab reflection. These should be submitted as a *PDF* on **Gradescope** for each team. Each reflection you complete is worth 20 points and will scale the performance part of your lab score (i.e., the 90 points excluding turning in your code) to be out of 70 or 50 points. For example, if you do both reflections, turn in your code on time, and get 70/90 for robot performance, your score will be 10 (for turning in code) + 40 (for doing both reflections) + 50*(70/90) = 89 points.

Pre-Lab Reflection

Task 1:

Write a short description of your robot design in terms of the components used and how they help accomplish the task. Examples include motors, sensors, locomotion type (wheels, tracks, legs, etc), or any other relevant components.

Task 2:

Draw a high-level flowchart of the robot's functionality.

Task 3:

Write the pseudocode that implements your flowchart.

Post-Lab Reflection

Task 1:

Write a short description of your robot's final design and how it differs from your initial concept. Why did you choose to make those changes?

Task 2:

Consider how your robot performed in the lab and how it was different from your expectations. Were there any particular shortcomings that you identified?

Task 3:

Explain what changes you would make to your design or implementation to achieve better results for the particular task.

Collaboration

You should feel free to discuss concepts in the course with other students in natural language (e.g., English). You should not share any programming code with others and must write all the robot's programming code yourself. If you access any sources other than the textbook or documents on Canvas, you must cite them in comments at the top of your code and send an email to the instructor with the citation. You must fully understand your code and be able to reproduce the algorithmic approach without references if asked. The Honor Code is in effect for these policies.

Good luck!